

Exercise 6 - Strip-mined element-wise multiplication of two arrays

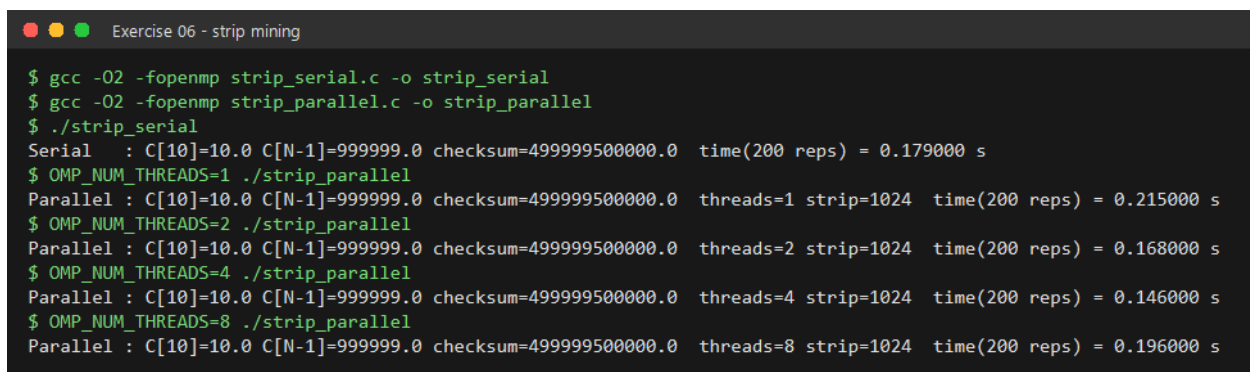
Source files : strip_serial.c, strip_parallel.c

(N = 1 000 000, C[i] = A[i] * B[i])

Build and execute

```
gcc -O2 -fopenmp strip_serial.c -o strip_serial && ./strip_serial
```

```
gcc -O2 -fopenmp strip_parallel.c -o strip_parallel && OMP_NUM_THREADS=4  
./strip_parallel
```



```
Exercise 06 - strip mining  
$ gcc -O2 -fopenmp strip_serial.c -o strip_serial  
$ gcc -O2 -fopenmp strip_parallel.c -o strip_parallel  
$ ./strip_serial  
Serial : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 time(200 reps) = 0.179000 s  
$ OMP_NUM_THREADS=1 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=1 strip=1024 time(200 reps) = 0.215000 s  
$ OMP_NUM_THREADS=2 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=2 strip=1024 time(200 reps) = 0.168000 s  
$ OMP_NUM_THREADS=4 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=4 strip=1024 time(200 reps) = 0.146000 s  
$ OMP_NUM_THREADS=8 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=8 strip=1024 time(200 reps) = 0.196000 s
```

How strip mining is applied

The original loop over 0..N-1 is rewritten as a pair of nested loops:

```
#define STRIP 1024           // multiple of the SIMD width  
  
#pragma omp parallel for schedule(static) // strips shared between threads  
for (int i = 0; i < N; i += STRIP) {  
    int end = (i + STRIP < N) ? i + STRIP : N; // clamp the last strip  
    #pragma omp simd           // strip itself is vectorised  
    for (int j = i; j < end; j++)  
        C[j] = A[j] * B[j];  
}
```

- * The outer loop steps through the array strip by strip, and it is these strips that OpenMP hands out to the different threads.
- * The inner loop handles a single strip. Because it is short and has a fixed trip count, the compiler can turn it into SIMD instructions.
- * A strip of 1024 floats is divisible by both the AVX2 width (8 floats) and the AVX-512 width (16 floats), so each strip fills complete vector registers. At 4 KB a strip also sits comfortably inside the L1 cache.

Measurements

The checksum (4999999500000) came out the same in every run. To obtain a measurable duration the kernel is executed 200 times. The speed-up is modest: a multiply that performs one FLOP per element is bound by memory bandwidth rather than by arithmetic, and two threads are already enough to saturate it.